◆ Assignment 05

Assignment 5 — Web Services, REST & XAML

Modules 8–9 · 10 points

 Deadline: Week 6 — Thursday, 11:59 PM

Scenario

You continue building the **Grades Prototype**. Today you fetch data from the web and build a simple UI bound to data — direct preparation for **Web API and MVC**.

This assignment covers: the **Request/Response** pattern, **REST** principles, and **XAML & data binding** basics.

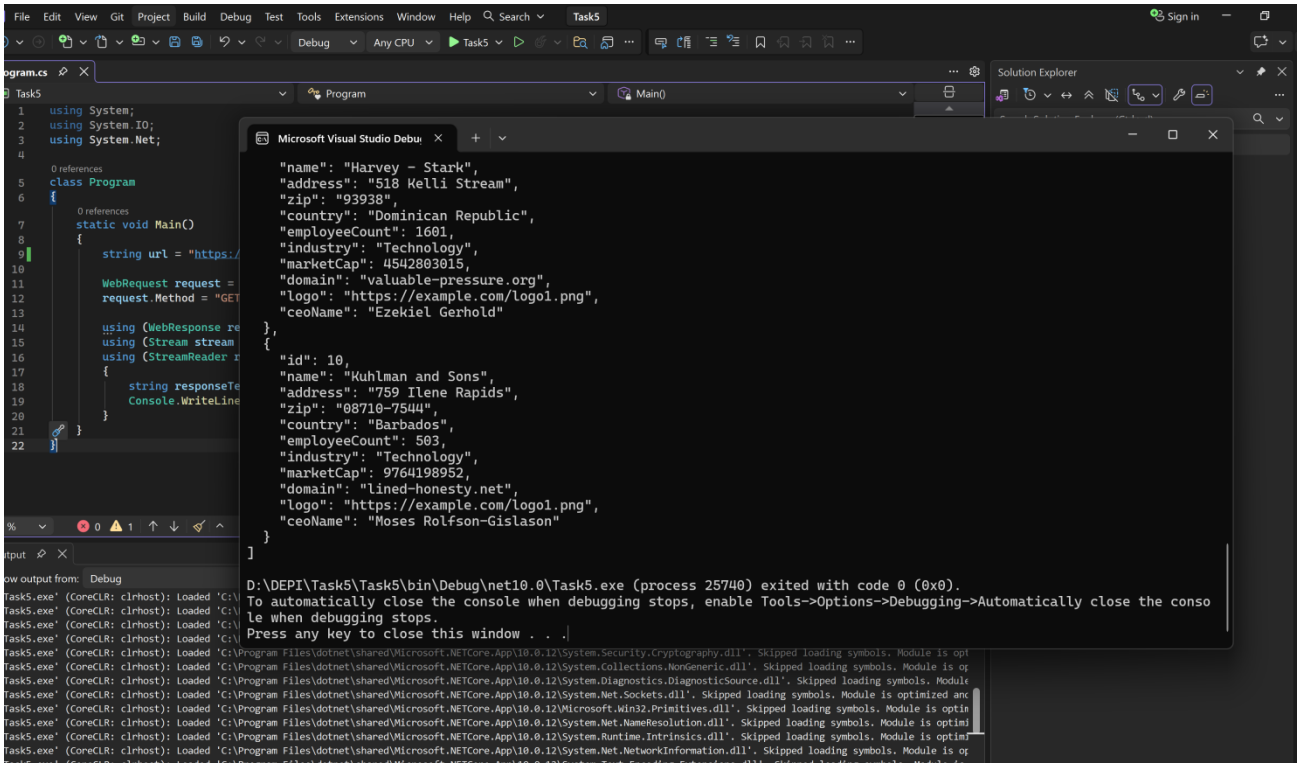
Part 1 — Web Connectivity (3 points)

1. Write code that creates a `WebRequest` to any public URL (e.g. `https://jsonplaceholder.typicode.com/todos/1`) and sets `Method = GET`.

```
using System;
using System.IO;
using System.Net;
class Program
{
   static void Main()
   {
      string url = "https://fake-json-api.mock.beeceptor.com/companies";

      WebRequest request = WebRequest.Create(url);
      request.Method = "GET";

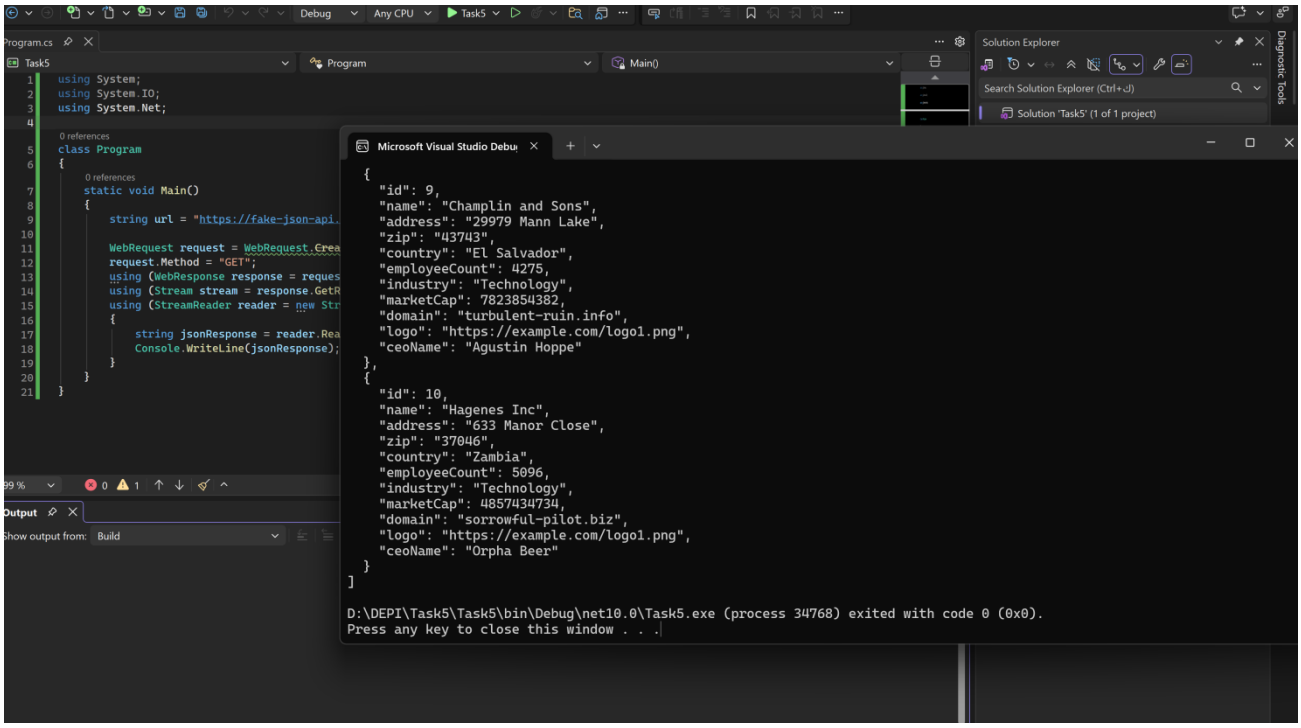
      using (WebResponse response = request.GetResponse())
      using (Stream stream = response.GetResponseStream())
      using (StreamReader reader = new StreamReader(stream))
      {
         string responseText = reader.ReadToEnd();
         Console.WriteLine(responseText);
      }
   }
}
```



2. Receive the `WebResponse` and read its content with `StreamReader`, then print the JSON.

```
using System;
using System.IO;
using System.Net;
class Program
{
   static void Main()
   {
      string url = "https://fake-json-api.mock.beeceptor.com/companies";

      WebRequest request = WebRequest.Create(url);
      request.Method = "GET";
      using (WebResponse response = request.GetResponse())
      using (Stream stream = response.GetResponseStream())
      using (StreamReader reader = new StreamReader(stream))
      {
         string jsonResponse = reader.ReadToEnd();
         Console.WriteLine(jsonResponse);
      }
   }
}
```



3. Deserialize the returned JSON into an object with `JsonSerializer.Deserialize` (links back to Session 4).

```
using System;
using System.IO;
using System.Net;
using System.Text.Json;
using System.Text.Json.Serialization;
public class Todo
{
   [JsonPropertyName("userId")]
   public int UserId { get; set; }

   [JsonPropertyName("id")]
   public int Id { get; set; }

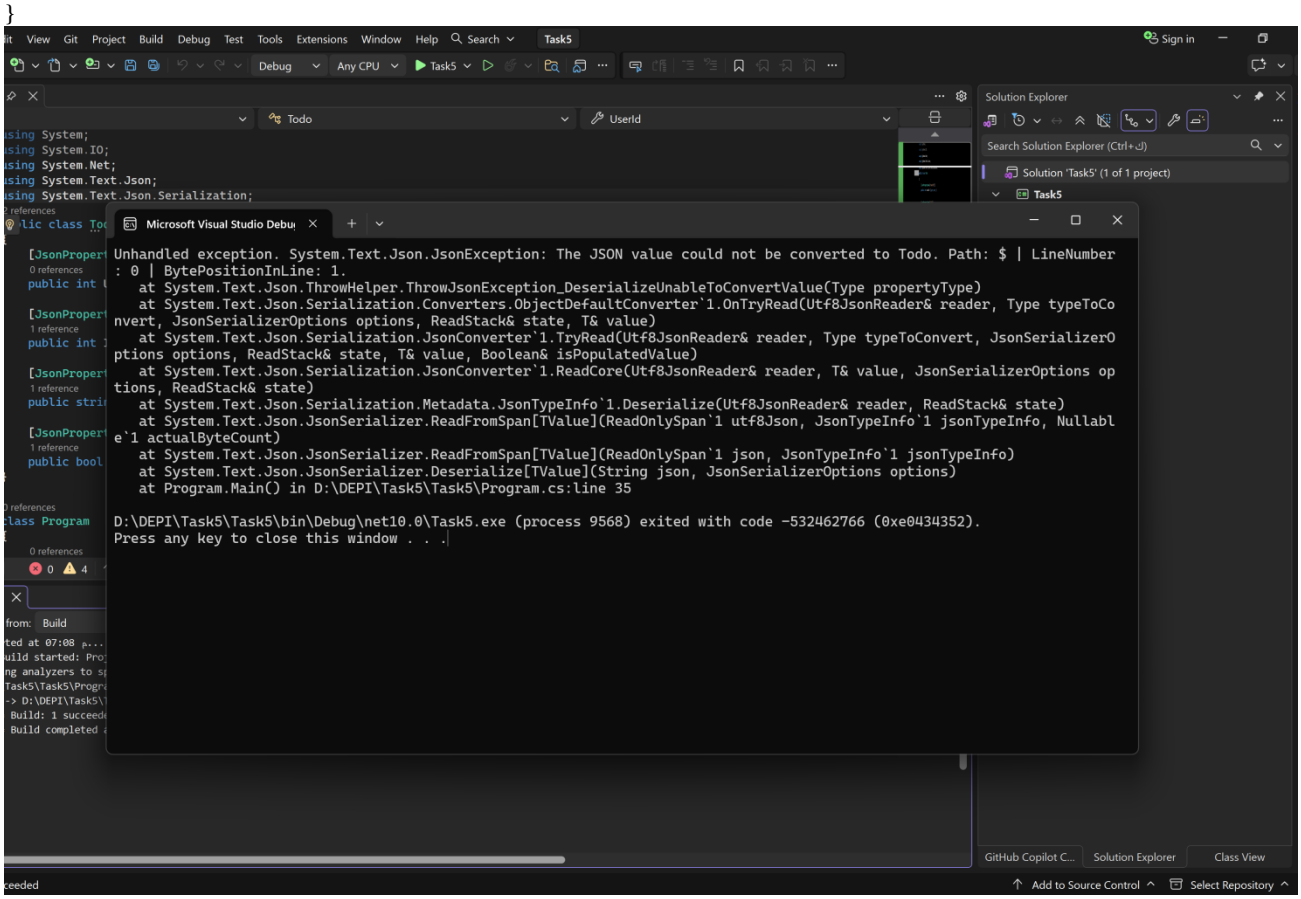
   [JsonPropertyName("title")]
   public string Title { get; set; }

   [JsonPropertyName("completed")]
   public bool Completed { get; set; }
}
class Program
{
   static void Main()
   {
      string url = "https://fake-json-api.mock.beeceptor.com/companies";

      WebRequest request = WebRequest.Create(url);
      request.Method = "GET";

      using (WebResponse response = request.GetResponse())
      using (Stream stream = response.GetResponseStream())
      using (StreamReader reader = new StreamReader(stream))
      {
         string jsonResponse = reader.ReadToEnd();
         Todo todo = JsonSerializer.Deserialize<Todo>(jsonResponse);
         Console.WriteLine($"ID: {todo.Id}");
         Console.WriteLine($"Title: {todo.Title}");
         Console.WriteLine($"Is Completed: {todo.Completed}");
      }
```

```
    }
}
```



Part 2 — REST (2 points)

Write a table (in a code comment or a text file) that shows:

4. Each HTTP verb (`GET`, `POST`, `PUT`, `DELETE`) and its meaning for a `/students` resource.

- **GET (`/students`)**: Retrieves student data. It only reads information and never modifies the database.

- **POST (`/students`)**: Creates a new student. Sends data in the request body to insert a new record.

- **PUT (`/students/{id}`)**: Updates an existing student completely. Replaces the entire student record with the new data provided.

- **DELETE (`/students/{id}`)**: Removes a student. Deletes the specified student record from the database.

5. Question: what is the difference between `GET /students` and `GET /students/5`?

- **`GET /students`**: Retrieves a **list of all students** from the database (a collection).

- **`GET /students/5`**: Retrieves the **specific details of a single student** whose ID is 5.

•

 Submission: .cs file + XAML text file + screenshots, posted to the group.

 Tip: **Working** code that you understand line by line is what matters — not perfect code. Use `breakpoints` if something doesn't run.